

7 DSA Patterns

That Solve 80% of LeetCode

● COMPLETE RESOURCE GUIDE ●

■ Sliding Window

■ ■ Two Pointers

■ ■ Fast & Slow Pointers

■ Binary Search

■ BFS / DFS on Trees & Graphs

■ Dynamic Programming

■ Backtracking

WHAT'S INSIDE

- ◆ When to recognize each pattern
- ◆ Copy-paste Python template per pattern
- ◆ Time & space complexity
- ◆ Core intuition explained simply
- ◆ 6–7 hand-picked LeetCode problems
- ◆ Pro tips & edge-case warnings

Made for [techbyshreyas](#) — [@techbyshreyas](#)

TABLE OF CONTENTS

01	Sliding Window Handles contiguous subarrays/substrings efficiently	O(n)
02	Two Pointers Eliminates nested loops with two indices moving toward each other	O(n)
03	Fast & Slow Pointers Floyd's tortoise-and-hare for cycle detection	O(n)
04	Binary Search Divide search space in half each iteration	O(n)
05	BFS / DFS on Trees & Graphs Systematic traversal for connectivity, shortest paths, and exploration	O(n)
06	Dynamic Programming Break overlapping subproblems; build bottom-up or top-down	O(n)
07	Backtracking Explore all choices; undo bad ones and try the next	O(n)

1

Sliding Window

Handles contiguous subarrays/substrings efficiently

WHEN TO USE THIS PATTERN

- Problem asks for longest/shortest subarray or substring with a condition
- You need to track a running sum, product, or frequency in a range
- Keywords: 'contiguous', 'subarray', 'substring', 'window of size k'

CORE INTUITION

Instead of recomputing from scratch for every subarray, maintain a window [left, right]. Expand right to include new elements; shrink from left when the window violates the constraint. This turns $O(n^2)$ brute force into $O(n)$.

PYTHON TEMPLATE

```
left = 0
window_state = {} # freq map, sum, etc.
result = 0

for right in range(len(arr)):
    # 1. Expand: add arr[right] to window
    window_state = update(window_state, arr[right])

    # 2. Shrink: while constraint broken
    while not valid(window_state):
        window_state = remove(window_state, arr[left])
        left += 1

    # 3. Update result
    result = max(result, right - left + 1)
```

PRACTICE PROBLEMS

Easy	LC 643	Maximum Average Subarray I
Medium	LC 3	Longest Substring Without Repeating Characters
Medium	LC 424	Longest Repeating Character Replacement
Medium	LC 567	Permutation in String
Hard	LC 76	Minimum Window Substring
Hard	LC 239	Sliding Window Maximum

COMPLEXITY

Time: $O(n)$

Space: $O(k)$

PRO TIP

- Fixed-size windows → simple;
- variable-size windows → two-pointer shrink loop.

2

Two Pointers

Eliminates nested loops with two indices moving toward each other

WHEN TO USE THIS PATTERN

- Sorted array + find pair/triplet with target sum
- Palindrome or symmetry checks
- Merging two sorted arrays or lists
- Keywords: 'sorted array', 'in-place', 'pair sum', 'reverse'

CORE INTUITION

Place one pointer at the start (left) and one at the end (right). Move them toward each other based on comparison logic. Works when the array is sorted or has a monotone property—moving one pointer is guaranteed to improve the condition.

PYTHON TEMPLATE

```
left, right = 0, len(arr) - 1
while left < right:
    current = arr[left] + arr[right]
    if current == target:
        # found answer
        break
    elif current < target:
        left += 1 # need bigger sum
    else:
        right -= 1 # need smaller sum
```

PRACTICE PROBLEMS

	LC	
Easy	125	Valid Palindrome
	LC	
Easy	344	Reverse String
Medium	LC 15	3Sum
Medium	LC 11	Container With Most Water
	LC	
Medium	167	Two Sum II – Sorted Array
Hard	LC 42	Trapping Rain Water

COMPLEXITY

Time: $O(n)$

Space: $O(1)$

PRO TIP

- For 3Sum, fix one element and run two-pointer on the rest. Sort first!

3

Fast & Slow Pointers

Floyd's tortoise-and-hare for cycle detection

WHEN TO USE THIS PATTERN

- Detect a cycle in a linked list or array
- Find the middle node of a linked list
- Find the entry point of a cycle
- Keywords: 'cycle', 'loop', 'circular', 'middle of list'

CORE INTUITION

Move slow pointer 1 step and fast pointer 2 steps per iteration. If a cycle exists, fast will eventually lap slow and they meet. After meeting, reset one pointer to head—they meet again exactly at the cycle entry.

PYTHON TEMPLATE

```
slow = fast = head
# Phase 1: detect cycle
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
    if slow == fast:
        break
    else:
        return None # no cycle
# Phase 2: find entry point
slow = head
while slow != fast:
    slow = slow.next
    fast = fast.next
# slow == fast == cycle start
```

PRACTICE PROBLEMS

Easy	LC 141	Linked List Cycle
Medium	LC 142	Linked List Cycle II
Easy	LC 876	Middle of the Linked List
Medium	LC 287	Find the Duplicate Number
Easy	LC 202	Happy Number

COMPLEXITY

Time: $O(n)$

Space: $O(1)$

PRO TIP

- To find the middle for 'reverse second half', stop slow when fast reaches end.

4

Binary Search

Divide search space in half each iteration

WHEN TO USE THIS PATTERN

- Sorted array; find target, first/last occurrence
- Answer is monotone (feasible/not feasible) — 'binary search on answer'
- Matrix with sorted rows/columns
- Keywords: 'sorted', 'rotated', 'minimum in rotated', 'search in matrix'

CORE INTUITION

Maintain $[lo, hi]$ inclusive bounds. At each step, compute mid and decide which half to discard. Key insight: binary search works on any monotone predicate, not just sorted arrays—you can binary search on the answer itself.

PYTHON TEMPLATE

```
lo, hi = 0, len(arr) - 1
while lo <= hi:
    mid = lo + (hi - lo) // 2 # avoids overflow
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        lo = mid + 1
    else:
        hi = mid - 1
return -1 # not found

# Binary search on answer pattern:
# lo, hi = min_possible, max_possible
# while lo < hi:
#     mid = (lo + hi) // 2
#     if feasible(mid): hi = mid
#     else: lo = mid + 1
```

PRACTICE PROBLEMS

Easy	LC 704	Binary Search
Medium	LC 33	Search in Rotated Sorted Array
Medium	LC 153	Find Minimum in Rotated Sorted Array
Medium	LC 74	Search a 2D Matrix
Hard	LC 4	Median of Two Sorted Arrays
Hard	LC 410	Split Array Largest Sum

COMPLEXITY

Time: $O(\log n)$

Space: $O(1)$

PRO TIP

■ Use $lo + (hi-lo)//2$ (not $(lo+hi)//2$) to prevent integer overflow in other languages.

5

BFS / DFS on Trees & Graphs

Systematic traversal for connectivity, shortest paths, and exploration

WHEN TO USE THIS PATTERN

- Shortest path in unweighted graph → BFS
- Explore all paths, detect cycles, topological sort → DFS
- Level-order traversal of trees → BFS
- Keywords: 'shortest', 'connected components', 'islands', 'path exists'

CORE INTUITION

BFS uses a queue and explores layer by layer — guarantees shortest path. DFS uses a stack (or recursion) and goes deep first — good for exhaustive search. Always track visited nodes to avoid infinite loops.

PYTHON TEMPLATE

```
from collections import deque

# BFS template
def bfs(graph, start):
    queue = deque([start])
    visited = {start}
    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# DFS template (iterative)
def dfs(graph, start):
    stack = [start]
    visited = set()
    while stack:
        node = stack.pop()
        if node in visited: continue
        visited.add(node)
        for neighbor in graph[node]:
            stack.append(neighbor)
```


PRACTICE PROBLEMS

Medium	LC 200	Number of Islands
Medium	LC 102	Binary Tree Level Order Traversal
Medium	LC 994	Rotting Oranges
Medium	LC 207	Course Schedule (Topo Sort)
Medium	LC 133	Clone Graph
Hard	LC 127	Word Ladder

COMPLEXITY

Time: $O(V+E)$

Space: $O(V)$

PRO TIP

■ For grids, treat each cell as a node; neighbors are 4-directional (or 8).

6

Dynamic Programming

Break overlapping subproblems; build bottom-up or top-down

WHEN TO USE THIS PATTERN

- Optimal substructure: optimal solution built from optimal sub-solutions
- Overlapping subproblems: same sub-problems solved repeatedly
- Keywords: 'maximum/minimum', 'count ways', 'can you achieve', 'longest'

CORE INTUITION

Define $dp[i]$ (or $dp[i][j]$) as the answer to a subproblem. Write a recurrence. Fill table bottom-up or memoize top-down. Common DP types: 1D (Fibonacci-like), 2D (grid paths, LCS), interval DP, knapsack, bitmask DP.

PYTHON TEMPLATE

```
# Top-down (Memoization)
from functools import lru_cache
@lru_cache(maxsize=None)
def dp(i, ...state...):
    if base_case: return base_value
    return min/max/sum(dp(i-1, ...), dp(i+1, ...))

# Bottom-up (Tabulation) - e.g. 0/1 Knapsack
dp = [[0] * (W+1) for _ in range(n+1)]
for i in range(1, n+1):
    for w in range(W+1):
        if weights[i-1] <= w:
            dp[i][w] = max(dp[i-1][w],
                           dp[i-1][w-weights[i-1]] + values[i-1])
        else:
            dp[i][w] = dp[i-1][w]
```

PRACTICE PROBLEMS

Easy	LC 70	Climbing Stairs
	LC	
Medium	322	Coin Change
	LC	
Medium	1143	Longest Common Subsequence
	LC	
Medium	300	Longest Increasing Subsequence
	LC	
Medium	416	Partition Equal Subset Sum
	LC	
Hard	312	Burst Balloons
	LC 10	
Hard		Regular Expression Matching

COMPLEXITY

Time: $O(n \cdot k)$ typically

Space: $O(n)$ or $O(n \cdot k)$

PRO TIP

■ Always start by asking: 'What is $dp[i]$ the answer to?' — defining state correctly is 80% of DP.

7

Backtracking

Explore all choices; undo bad ones and try the next

WHEN TO USE THIS PATTERN

- Generate all subsets, permutations, combinations
- Constraint satisfaction: N-Queens, Sudoku, Word Search
- Keywords: 'all possible', 'generate', 'find all', 'combinations'

CORE INTUITION

Build the solution incrementally. At each step, try every valid choice. If a choice leads to a dead end, undo it (backtrack) and try the next. Prune early to avoid exploring impossible branches.

PYTHON TEMPLATE

```
def backtrack(start, current, result):
    # Base case: valid complete solution
    if is_complete(current):
        result.append(list(current))
        return
    for choice in get_choices(start):
        if not is_valid(choice, current):
            continue # prune
        current.append(choice) # choose
        backtrack(start + 1, current, result) # explore
        current.pop() # unchoose (backtrack)
    result = []
    backtrack(0, [], result)
```

PRACTICE PROBLEMS

Medium	LC 78	Subsets
Medium	LC 46	Permutations
Medium	LC 39	Combination Sum
Medium	LC 79	Word Search
Hard	LC 51	N-Queens
Hard	LC 37	Sudoku Solver

COMPLEXITY

Time: $O(2^n)$ or $O(n!)$

Space: $O(n)$ recursion depth

PRO TIP

- Sort input first to easily skip duplicates: if $i > \text{start}$ and $\text{nums}[i] == \text{nums}[i-1]$: continue

QUICK RECOGNITION CHEAT SHEET

Read the problem → spot the keyword → apply the pattern

Sliding Window

[contiguous subarray] [window of size k] [longest substr] [minimum subarray]

Two Pointers

[sorted array] [pair sum] [palindrome] [reverse in-place] [container]

Fast & Slow Pointers

[linked list cycle] [middle of list] [loop detection] [duplicate number]

Binary Search

[sorted array] [rotated sorted] [search in matrix] [minimum in rotated] [feasibility]

BFS / DFS

[number of islands] [shortest path] [connected components] [level order]

Dynamic Programming

[max/min value] [count ways] [can you achieve] [longest increasing] [knapsack]

Backtracking

[all subsets] [all permutations] [combination sum] [N-Queens] [word search]

Follow @techbyshtreyas for more DSA content • Practice daily • Consistency beats intensity